

EE6407 Genetic Algorithms and Machine Learning

Quiz 1 Preparation Notes: Evolutionary Computing Part 1 (Weeks 1–4)

Living course note

29 August 2026

Scope and sources. This is the detailed, self-contained teaching note for Quiz 1. It covers the examinable scope: problem classification, computational complexity, evolutionary-computing foundations, the evolutionary-algorithm loop and components, 8-queens, the simple genetic algorithm, optimisation context and no free lunch, and — added after the Week 3 lecture — the five representations and the variation operators that suit each of them. The authoritative source for the first part is `../resources/lecture-01-evolutionary-computing-part-1.pdf`, PDF pp. 1–36, supported by the Week 1 to Week 3 transcripts. The Week 3 lecture worked from a second slide deck that is not available locally, so `../resources/week-03-transcript.txt` is the controlling source for Sections 6 and 7; standard formulas there are reconstructed and flagged as such. Historical questions are used only as pattern evidence; every worked answer here is independently derived.

Contents

1	Evolutionary computing in one idea	1
2	Classifying the problem before choosing a method	1
3	Why search can become difficult	2
4	The simplified evolutionary metaphor	2
5	Foundations checkpoint	2
6	Where evolutionary algorithms stand	3
6.1	Heuristics, metaheuristics, and the two inspirations	3
6.2	Deterministic versus heuristic problem solving	3
6.3	Building the picture that leads to no free lunch	3
6.4	Global search, local search, and what a neighbourhood is	4
6.5	Checkpoint	4
7	Representations and their variation operators	5
7.1	Why representation is the first design decision	5
7.2	The five representations at a glance	5
7.3	Binary representation	5
7.4	Integer representation	7
7.5	Real-valued (floating-point) representation	7
7.6	Permutation representation	10
7.7	Tree representation	12
7.8	Compatibility summary	13
7.9	Checkpoint	13

8 Quiz 1 scope and evidence	14
8.1 What is confirmed, and what is only supporting evidence?	14
9 One-page memory core	17
10 Worked archetypes	19
10.1 Archetype 1: classify the problem before choosing an algorithm	19
10.2 Archetype 2: reason correctly about P, NP, NP-complete, and NP-hard	19
10.3 Archetype 3: trace the evolutionary algorithm loop	19
10.4 Archetype 4: represent and score 8-queens	20
10.5 Archetype 5: calculate one complete SGA generation	20
10.6 Archetype 6: choose a representation and legal operators	20
10.7 Archetype 7: real-valued crossover arithmetic	21
10.8 Archetype 8: permutation crossover under exam conditions	21
10.9 Archetype 9: binary encoding of a real variable	21
11 Progressive drills	22
12 Solutions to the progressive drills	24
13 Mock Quiz A: foundations and a hand-worked generation	25
14 Mock Quiz A: marking guide	26
15 Mock Quiz B: representation, operators, and optimisation context	27
16 Mock Quiz B: marking guide	28
17 Seven-day preparation schedule	29
17.1 Quiz-day checklist	29

1 Evolutionary computing in one idea

Evolutionary computing (EC) is a family of methods for searching for good solutions by maintaining and improving a population of candidates. Genetic algorithms (GAs) are an important member of this family. In lecture, the names *evolutionary computing*, *evolutionary algorithm*, and *genetic algorithm* may be used informally, but strictly a GA is one specific type of evolutionary algorithm.

EC is useful when a problem has a large, irregular, discrete, mixed, or non-differentiable search space, or when exact optimisation would be too expensive. It does not automatically guarantee the best possible answer. It is a stochastic method for finding high-quality candidates within a computational budget.

2 Classifying the problem before choosing a method

Represent a system abstractly as $y = M(x)$:

- If M and desired output y are known but x is unknown, we have an **optimisation** problem.
- If input–output pairs are known but M is unknown, we have a **modelling** or learning problem.
- If x and M are known but y is unknown, we have a **simulation** problem.

Many machine-learning tasks can be written as optimisation: select model parameters to minimise prediction error. The distinction is still useful because it clarifies what information is available and what the method must find.

The *search space* S contains all candidates the algorithm may consider. The *feasible set* $\mathcal{F} \subseteq S$ contains those satisfying every hard constraint. An objective function $f : S \rightarrow \mathbb{R}$ measures quality. A constraint-satisfaction problem asks for any member of $\mathcal{F}$; a constrained optimisation problem asks for the best member.

3 Why search can become difficult

The number of candidate solutions can grow much faster than intuition suggests. For a route through n labelled cities, there are $n!$ possible orders before considering symmetries. Exhaustive enumeration is therefore impractical even at modest n .

Complexity words provide context. A decision problem is in **P** if it can be solved in polynomial time. It is in **NP** if a proposed yes-answer can be verified in polynomial time. NP-complete problems are the hardest known problems in NP under polynomial reductions, and NP-hard problems are at least as hard but need not be decision problems. Whether **P** = **NP** is unresolved. These labels motivate heuristic search; they do not prove that an EA will solve a particular instance well.

4 The simplified evolutionary metaphor

Evolutionary algorithms borrow four ideas from a simplified account of natural evolution: resources are limited, individuals differ, some differences affect reproductive success and are heritable, and random variation supplies new differences. In computing, *fitness* is a deliberately designed numerical score rather than a biological fact.

The terms below connect the problem to its representation:

- A *phenotype* is the candidate solution in the original problem domain.
- A *genotype* is the encoding manipulated by the algorithm.
- A *chromosome* is a complete genotype; a *gene* is one component; an *allele* is a possible value of that component.

The representation is an engineering decision. It should make valid solutions reachable and should work naturally with the variation operators introduced later. A poor representation can make a simple problem unnecessarily difficult.

5 Foundations checkpoint

- Can I distinguish optimisation, modelling, and simulation using $y = M(x)$?
- Can I identify a search space, a feasible set, and an objective?
- Can I explain why combinatorial spaces make exhaustive search difficult?
- Can I distinguish evolutionary computing from a genetic algorithm?
- Can I explain genotype, phenotype, chromosome, gene, and allele?

Source: `../resources/lecture-01-evolutionary-computing-part-1.pdf`, PDF pp. 2–16;
`../resources/week-01-transcript.txt`..

6 Where evolutionary algorithms stand

6.1 Heuristics, metaheuristics, and the two inspirations

A *heuristic* is a rule of thumb: a rule that decides what to try next without promising that the choice is optimal. A *metaheuristic* sits one level above that. It is a general search scheme that organises such rules, and in the lecture’s phrasing it is “a heuristic about heuristics”. The point the lecturer stresses is the source of the rules: a genetic algorithm is a metaheuristic whose guiding rules are borrowed from *nature* rather than from a human expert’s knowledge of one particular problem. Ant-colony optimisation, particle-swarm optimisation, and related nature-inspired schemes are metaheuristics in exactly the same sense.

Two separate strands of natural inspiration are worth being able to name separately:

- **Darwinian natural evolution.** Resources are insufficient, so there is competition; selection retains the fitter individuals; and reproduction is the *reward* for having survived selection.
- **Genetics.** DNA supplies the metaphor of an encoded description (the genotype) whose physical expression is the organism (the phenotype).

6.2 Deterministic versus heuristic problem solving

Before evolutionary methods, the standard tools were exact deterministic searches such as branch and bound. The two families fail in opposite ways.

	Deterministic (branch and bound)	Heuristic / metaheuristic (GA)
Guarantee	Finds the optimum if run to completion.	No guarantee that the returned solution is optimal.
Run time	Bounded in principle, but on a hard combinatorial instance the bound can exceed a lifetime.	No useful worst-case bound; in practice a budget of generations or evaluations stops it.
Repeatability	Same run, same answer.	Stochastic: different runs give different answers.
Practical failure	It converges, but a run you can actually afford may stop far from the best solution.	It returns a reasonable solution, but cannot certify it.
Lecture slogan	“I do not care whether it <i>works</i> , as long as it <i>converges</i> .”	“I do not care whether it <i>converges</i> , as long as it <i>works</i> .”

This is the practical reason NP-hard problems pushed the field towards heuristics: an exact method that is guaranteed but unaffordable is not a usable method.

6.3 Building the picture that leads to no free lunch

The lecture derives “no free lunch” from a picture rather than from the formal proof, and the picture is the examinable part. Draw performance on the vertical axis and *all possible problems* on the horizontal axis, in some arbitrary but fixed order.

1. **Random search.** A low, flat line. It is never good, but it is never specifically bad either, because it assumes nothing.
2. **A method tailored to one problem.** A tall, narrow spike. It is excellent on the class it was customised for, and poor elsewhere.

3. **A classical EA with no problem knowledge.** A broad, moderately high curve: generally better than random search over a wide range of problems, and generally worse than a custom method on that method's own problem.
4. **EAs with increasing domain knowledge (EA₁ to EA₄).** Each increment of knowledge – a better representation, a repair rule, a specialised operator, a seeded population – pushes the curve *up* on the intended class and *down* elsewhere. The curve is deformed, not lifted.

Now ask the lecturer's question. If you want to solve one specific problem, the most specialised algorithm whose peak sits over that problem is clearly the best choice. But what is the *average* performance of each curve across *all* problems? If you accept the picture – each algorithm redistributes a fixed amount of performance rather than creating more – then every curve encloses the same area, so every algorithm has the same average.

The statement. This is the *no free lunch* (NFL) theorem of optimisation: averaged over all possible objective functions, no search algorithm outperforms any other. Gains on one class of problems are paid for by losses on another. The result is due to Wolpert and Macready (1997); the lecture refers to it as “the classic paper in optimisation”.

What NFL does and does not say. It does *not* say that algorithm choice is irrelevant. Real engineering problems are a tiny, structured subset of “all possible objective functions”, and on that subset matching the algorithm's bias to the problem's structure is exactly what works. NFL says there is no universally best optimiser, so the honest question is never “which algorithm is best?” but “which algorithm is biased in a way that suits *this* problem?”

6.4 Global search, local search, and what a neighbourhood is

Global optimisation means searching over all possibilities with the best solution in the whole feasible set as the objective. Stating that as the objective is not the same as achieving it: a global search may still return a local optimum.

Local search means looking only in the vicinity of the current solution: “where do I go next?” is answered by inspecting neighbours. The lecture's warning is that “neighbour” is not a fixed notion. Depending on the design it can mean

- a small difference in gene values, such as a single changed allele;
- closeness after decoding, that is, similarity in the phenotype;
- a distance measure declared in advance, such as Hamming distance on bit strings.

Consequently *local optimum* is meaningless until the neighbourhood is stated. A point can be a local optimum under one-bit flips and not under two-bit flips. Whenever a question asks about local optima, name the neighbourhood first.

6.5 Checkpoint

- Can I define heuristic and metaheuristic and say why a GA is the latter?
- Can I name the two strands of natural inspiration separately?
- Can I sketch the four performance curves and read the NFL conclusion off them?
- Can I state what NFL does *not* claim?
- Can I explain why “local optimum” requires a neighbourhood definition?

Source: `../resources/week-03-transcript.txt`, opening recap and the “EA in context” segment; consistent with `../resources/lecture-01-evolutionary-computing-part-1.pdf`, PDF pp. 34–36..

7 Representations and their variation operators

7.1 Why representation is the first design decision

Representation is the bridge between the problem as it exists in the world and the strings the algorithm can manipulate. It comes first because everything downstream depends on it: the size and shape of the search space, what counts as a valid candidate, how fitness is computed, and – above all – *which variation operators are even legal*.

The lecture’s rule is short and is worth memorising verbatim: **the operators must suit the representation**. A crossover that works perfectly on bit strings can destroy a permutation. The purpose of both operators is the same in every representation – create variation, and hence diversity – but the mechanism must respect the encoding.

7.2 The five representations at a glance

Type	Genotype space	Typical problems	Characteristic operators
Binary	$\{0, 1\}^L$	Anything encodable as bits; the original GA setting	Bit-flip mutation; one-point, multi-point, uniform crossover
Integer	$\{1, \dots, K\}^L$	Labelling and assignment, e.g. graph colouring	Creep or random-resetting mutation; binary-style crossovers
Real-valued	$\mathbb{R}^n$ (usually boxed)	Function optimisation, control gains, filter design	Uniform or Gaussian mutation, self-adaptation; arithmetic and blend crossover
Permutation	orderings of $1, \dots, L$	Sequencing and routing: TSP, N -queens, scheduling	Swap, insert, scramble, inversion; order-1, PMX, cycle, edge recombination
Tree	expression trees	Evolving programs, formulas, structures (genetic programming)	Subtree mutation; subtree exchange crossover

7.3 Binary representation

Encoding and decoding. Holland’s 1975 genetic algorithm was demonstrated mainly on bit strings, and a great many problems were solved this way. *Encoding* maps the phenotype space (the problem) into the genotype space $\{0, 1\}^L$, where L is the string length. *Decoding* is the inverse map, applied before every fitness evaluation.

Mutation: bit flip. Mutation is a *unary* operator: it takes one chromosome and changes it. Walk along the string gene by gene; at each gene, toss a coin that comes up “mutate” with probability p_m ; if it does, flip that bit. In binary this is unambiguous, because the only other value is the complement: $0 \rightarrow 1$ and $1 \rightarrow 0$.

How large should p_m be? The lecture’s guide is that the mutation rate is *small*, typically between $1/\mu$ (one over the population size) and $1/L$ (one over the chromosome length). The reason is easy to state: at a large rate, say $p_m = 0.8$, you are flipping most bits every generation, which is indistinguishable from random search. Mutation is meant to inject randomness *sporadically*, not continuously.

A subtlety: mutation is not as unbiased as it looks. Suppose the bit string decodes to an integer. Flipping the most significant bit changes the decoded value enormously; flipping the least significant bit barely moves it. So although every gene is chosen with the same probability, the *effect* of mutation is heavily biased towards the early, more significant genes. If that bias is a problem, the standard remedy named in the lecture is **Gray coding**, in which numerically adjacent values differ by exactly one bit, so a single flip is always a small phenotype move.

Crossover. Crossover (recombination) is the operator that implements inheritance: it combines material from two parents that selection has already judged to be comparatively fit.

- **One-point.** Choose one crossover site, split both parents there, and build children from complementary halves.
- **Multi-point.** Choose n sites, split into $n + 1$ segments, and take segments alternately from the two parents. Useful for long strings, where a single cut is a very coarse operation.
- **Uniform.** Decide gene by gene, at random, which parent contributes.

Nothing guarantees a child is fitter than its parents; the hope is that useful partial structures occasionally combine, and selection keeps the successes.

Do we need both operators? This was an open debate in early GA research. The settled answer in the lecture is that both are needed, and the cleanest argument is about information:

Only crossover can combine information from two parents; only mutation can introduce new information.

The allele-frequency argument. Here is the lecture's concrete demonstration that crossover alone is not enough. Start a binary population in which zeros and ones occur equally often, a ratio of 1 : 1 over the whole population. Crossover only *moves* existing genes between strings; it never creates a value that was not already present at that locus. So after ten generations, or a hundred, of crossover alone, the population-wide ratio of zeros to ones is still 1 : 1. If some allele has been lost entirely, crossover can never recover it. Mutation can, because it manufactures values rather than shuffling them.

Exploration and exploitation (lecture convention). Searching a landscape needs two opposing forces: *exploration*, covering new regions, and *exploitation*, refining what already looks good. The lecture assigns them as follows:

- **Crossover is explorative.** It makes a large jump to a point somewhere “between” the two parents – the lecture's analogy is interpolation.
- **Mutation is exploitative.** It takes an existing solution and perturbs it slightly, hoping the small variation is an improvement.

So under this convention, the advice for a GA that is under-performing is to adjust the balance – typically by raising the mutation rate to get more local refinement, or the crossover rate for more coverage.

Caution, worth knowing but not worth arguing in the quiz. Several standard textbooks assign these labels the other way round, calling mutation the explorative operator (it can reach anywhere) and crossover the exploitative one (it recombines material already in the population). Use the lecture's convention above in this course, and if a question asks you to justify the assignment, argue from *step size*: crossover makes the large jump, mutation the small local move.

7.4 Integer representation

When to use it. Instead of $\{0, 1\}$, each gene draws from a finite set of K labels, $\{1, 2, \dots, K\}$, giving a genotype space $\{1, \dots, K\}^L$. This suits problems whose decisions are categorical: assigning one of three colours, sorting items into classes, choosing a machine for a job.

Mutation. With more than two values, “flip it” is no longer defined, so there are two standard choices.

- **Creep mutation.** Add a small positive or negative amount to the gene with probability p . It is a small, local move – and that is also its weakness. Because every step is small, individuals tend to *creep* towards one another over the generations, and the population loses diversity. This is the mechanism of **premature convergence**: the run settles before it has explored enough.
- **Random resetting.** With some probability, replace the gene by a value drawn uniformly from $\{1, \dots, K\}$. This is the true analogue of the binary bit flip, because binary mutation is always a complete change of value, never a small step.

Crossover. Everything from the binary case carries over unchanged: one-point, multi-point, and uniform crossover all produce valid integer strings, because any label is legal at any position.

Worked example: graph colouring. Imagine you must print a map of the United States so that the regions are distinguishable. Adjacent regions must therefore get different colours. Abstract each region to a *vertex* and each shared border to an *edge*, giving a graph $G = (V, E)$ with $|V| = L$ regions. A colouring assigns one colour to each vertex such that adjacent vertices differ.

With k available colours, the encoding is immediate: a chromosome is

$$c = (c_1, c_2, \dots, c_L), \quad c_i \in \{1, \dots, k\},$$

where c_i is the colour of region i , so the genotype space is $\{1, \dots, k\}^L$.

The objective function counts broken adjacency rules:

$$V(c) = \sum_{(u,v) \in E} \mathbf{1}[c_u = c_v],$$

which is to be minimised, with $V(c) = 0$ exactly at a valid colouring. For a maximisation convention use $f(c) = |E| - V(c)$. Note what the encoding does *not* do: unlike the N -queens permutation, it cannot make the constraint impossible to violate, so the constraint must be paid for in the objective.

The associated question is the k -colouring question: what is the minimum k such that no two adjacent regions share a colour? Two facts from the lecture:

- $k = 3$ is *not* enough for the map of the United States.
- $k = 4$ is enough for any such map – the four-colour theorem, proved for planar maps.

A good self-test is to take the map of Singapore’s regions and check by hand whether three colours suffice.

Source: ../resources/week-03-transcript.txt, representation segment..

7.5 Real-valued (floating-point) representation

When to use it. Many engineering problems ask for continuous parameter values: the gains of a PID controller, the coefficients of a filter, the shape parameters of a design. If the answer is a vector of real numbers, the natural encoding is a vector of real numbers, $x = (x_1, \dots, x_n) \in \mathbb{R}^n$, usually with each variable boxed into an interval.

Benchmark functions. The GA community demonstrates robustness on *benchmark* functions: standard, published test problems $f : \mathbb{R}^n \rightarrow \mathbb{R}$ with known optima, so that different algorithms can be compared. The lecture's example is the **Ackley function**, a multimodal benchmark whose global minimum is at the origin with value 0:

$$f(x) = -20 \exp\left(-0.2 \sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2}\right) - \exp\left(\frac{1}{n} \sum_{i=1}^n \cos(2\pi x_i)\right) + 20 + e.$$

Here n is the *dimension* – the number of variables being solved for. At $n = 2$ it can be plotted as a surface, with the two variables horizontal and $f(x)$ vertical, and you can see the single global basin surrounded by many local wells. Benchmarks are deliberately scalable: researchers run the same function at $n = 10, 100$, or 1000 to show an algorithm survives high dimension.

You may still use binary for a real problem. Real-valued encoding is natural, but it is not compulsory. To encode a bounded real variable $z \in [x, y]$ as a bit string $a = (a_1, \dots, a_L)$, decode with

$$\Gamma(a) = x + \frac{y - x}{2^L - 1} \sum_{i=1}^L a_i 2^{L-i}.$$

The important structural point: you are mapping an *infinite* space (the real interval) onto a *finite* one (2^L strings). L therefore sets the *resolution*. A longer string gives finer accuracy but a larger space and slower evolution, so resolution is a design trade-off, not a free improvement.

Worked conversion. Take the interval $[0.25, 1.88]$ and $L = 4$, and decode $a = 1010$.

$$\sum a_i 2^{4-i} = 8 + 2 = 10, \quad \Gamma(a) = 0.25 + \frac{1.88 - 0.25}{2^4 - 1} \times 10 = 0.25 + \frac{1.63}{15} \times 10.$$

Since $1.63/15 = 0.108\bar{6}$, we get $\Gamma(a) = 0.25 + 1.0867 = 1.3367$ to four decimal places. The step between adjacent codes is 0.1087 ; going to $L = 10$ would shrink it to $1.63/1023 \approx 0.00159$.

Mutation in a real vector. Flipping is meaningless here, so mutation must *generate a new value*.

- **Uniform (random resetting).** Replace x_i by a value drawn uniformly from $[LB, UB]$. This is the direct analogue of a bit flip or an integer reset.
- **Gaussian (non-uniform).** Perturb the current value,

$$x'_i = x_i + N(0, \sigma),$$

where σ , the standard deviation, acts as the *mutation step size*. Small σ gives a fine local move; large σ approaches a random reset.

Self-adaptive mutation, and why the order matters. Rather than fixing σ by hand, put it *inside the chromosome*:

$$\langle x_1, \dots, x_n, \sigma \rangle.$$

The step size then evolves along with the solution, and the algorithm tunes its own search scale. Two things must now be mutated, and the lecture insists on a specific order:

Mutate σ first, then use the new σ' to mutate x .

The rationale is worth reconstructing rather than memorising. Fitness is evaluated on x , never on σ ; σ has no quality of its own. So σ can only be judged *indirectly*, through the quality of the offspring it produced. Mutating σ first means the x' that gets evaluated is the one that σ' actually generated, so a good fitness is evidence that σ' was a good step size. Do it the other way round and the evaluated x' owes nothing to σ' , so the fitness says nothing about it. In short: x' is the primary evaluation, σ' is judged secondarily through it, and that dependency dictates the order.

Crossover: arithmetic recombination. For real values the natural way to inherit from both parents is to interpolate. Choose $\alpha \in [0, 1]$ and set, for the genes being crossed,

$$z_i = \alpha x_i + (1 - \alpha)y_i.$$

The value of α may be held fixed (0.5 is the common default), varied with the age of the population, or drawn at random each time. Three variants differ only in *how many genes* are treated this way:

- **Single arithmetic:** blend at one randomly chosen gene; all other genes are copied unchanged.
- **Simple arithmetic:** choose a crossover point and blend every gene after it.
- **Whole arithmetic:** blend every gene.

Worked example. Let $x = (0.3, 0.7, 0.8, 0.2)$ and $y = (0.5, 0.1, 0.2, 0.9)$.

Whole arithmetic, $\alpha = 0.3$:

$$z^{(1)} = 0.3x + 0.7y = (0.44, 0.28, 0.38, 0.69), \quad z^{(2)} = 0.7x + 0.3y = (0.36, 0.52, 0.62, 0.41).$$

A useful check: $z_i^{(1)} + z_i^{(2)} = x_i + y_i$ for every i , here $(0.8, 0.8, 1.0, 1.1)$.

Whole arithmetic, $\alpha = 0.5$: both children equal $(0.40, 0.40, 0.50, 0.55)$. This is a trap worth noticing: at $\alpha = 0.5$ the two arithmetic children are identical, so the operator has produced one point, not two.

Simple arithmetic after gene 2, $\alpha = 0.5$: $z^{(1)} = (0.3, 0.7, 0.50, 0.55)$ and $z^{(2)} = (0.5, 0.1, 0.50, 0.55)$.

The limitation of arithmetic crossover. Ask what values an arithmetic child can take. Since $z_i = \alpha x_i + (1 - \alpha)y_i$ with $\alpha \in [0, 1]$, the child always lies *between* the parents, whatever α is. Over a whole chromosome, the offspring are confined to the bounding box spanned by the two parents. The operator can therefore contract the population but never expand it, which is a real cause of premature convergence.

Blend crossover (BLX- α). The fix is to let the offspring reach slightly *outside* the parents. Assume for a gene that $x_i < y_i$, so the parental distance

$$d_i = y_i - x_i$$

is positive. Sample the child value uniformly from the widened interval

$$z_i \in [x_i - \alpha d_i, y_i + \alpha d_i].$$

The square brackets denote the sampling range, and α controls how far beyond the parents the search may reach; it can be made adaptive as the run progresses.

Comparison at one gene. With $x_i = 0.3$, $y_i = 0.7$, we have $d_i = 0.4$. Arithmetic crossover can only produce a value in $[0.3, 0.7]$. Blend crossover with $\alpha = 0.5$ samples from $[0.3 - 0.2, 0.7 + 0.2] = [0.1, 0.9]$ – twice the width, and genuinely explorative.

Multi-parent recombination. Nothing in the mathematics requires exactly two parents; the restriction is inherited from the biological metaphor, not from the problem. The lecture’s reminder is that the goal is to solve the problem, and nature is an inspiration rather than a constraint. Two extensions are named:

- **Diagonal crossover (“simple type 1”).** With p parents, choose $p - 1$ crossover points and build each child by taking segments *diagonally* across the parents, wrapping around to the first parent at the end. This generalises one-point crossover to p parents and yields p children.
- **Multi-parent arithmetic.** Extend the arithmetic blend to a weighted average of p parents.

Source: ../resources/week-03-transcript.txt, real-valued representation segment. The Ackley formula and the four-decimal conversion above are standard results reconstructed here, not transcribed from the audio..

7.6 Permutation representation

When to use it. A large family of problems is about *ordering* or *sequencing* rather than about values. The travelling-salesman problem asks for a sequence visiting every city once and only once, at minimum total distance; N -queens, scheduling, and assembly ordering have the same character. Here a chromosome is a permutation: each of the values $1, \dots, L$ appears exactly once.

Two payoffs from the lecture:

- **TSP with 30 cities:** encode as a permutation of $1, \dots, 30$, where the gene *order* is the travel order. Finding the optimum is NP-hard, which is exactly why a metaheuristic is wanted.
- **N -queens:** a permutation of $1, \dots, 8$ automatically places one queen per row and one per column, so those two constraints cannot be violated. Only the diagonal conflicts remain to be optimised. This is representation doing the work that a penalty term would otherwise have to do.

The governing rule. Every operator must return a valid permutation. Ordinary one-point crossover does not: it duplicates some values and omits others. Permutation operators are therefore built specifically to preserve the “once and only once” property.

Mutation on a permutation. Mutation is still unary. Four standard forms, from most to least disruptive of adjacency:

- **Swap.** Pick two positions and exchange their values. Note the logic: you cannot simply overwrite one gene with a new value, because that value already occurs elsewhere – so you must also move the one it displaced.
- **Insert.** Pick two genes and move one so that it sits next to the other, shifting the rest along.
- **Scramble.** Pick a range of genes and shuffle them randomly.
- **Inversion.** Pick a range and reverse it. This is the gentlest: it changes the order but *preserves most adjacency information*, because only the two boundary links are broken.

Crossover on a permutation. Four operators are covered, and all four are illustrated below with the same pair of parents used in the lecture:

P_1	1	2	3	4	5	6	7	8	9
P_2	9	3	7	8	2	6	5	1	4

In each case the chosen segment is positions 4–7, so P_1 contributes 4 5 6 7 and P_2 contributes 8 2 6 5.

1. Order-1 crossover. Copy the segment from P_1 into the child at the same positions. Then fill the remaining positions from P_2 , taking P_2 ’s values *in P_2 ’s order*, starting immediately after the segment and wrapping around, skipping any value already present.

Starting from position 8, P_2 reads 1, 4, 9, 3, 7, 8, 2, 6, 5. Discarding those already in the child (4, 5, 6, 7) leaves 1, 9, 3, 8, 2, which fill positions 8, 9, 1, 2, 3:

$$\text{child} = 3 \ 8 \ 2 \mid 4 \ 5 \ 6 \ 7 \mid 1 \ 9.$$

One parent supplies explicit values; the other supplies relative order.

2. Partially mapped crossover (PMX). PMX is the most commonly used permutation crossover. Steps:

1. Copy a random segment from P_1 into the child at the same positions.

2. Look at the values in the *same* segment of P_2 . Some already appear in the copied segment – these are the *partially matched* ones and need no action. For each value v that does not, follow the mapping chain to find its home: look at the value w that P_1 holds at v 's position, find where w sits in P_2 , and place v there if the slot is free; if not, repeat with w .
3. Fill every remaining position with P_2 's value at that position.

Worked, child 1. Segment gives – – – 4 5 6 7 – –. P_2 's segment is 8, 2, 6, 5; the 6 and 5 are already present, so only 8 and 2 must be placed.

- 2 sits at position 5; P_1 has 5 there; 5 sits at position 7 in P_2 ; position 7 is occupied by 7; 7 sits at position 3 in P_2 ; position 3 is free, so 2 goes to position 3.
- 8 sits at position 4; P_1 has 4 there; 4 sits at position 9 in P_2 ; position 9 is free, so 8 goes to position 9.

Filling positions 1, 2 and 8 from P_2 gives 9, 3, 1:

$$\text{child}_1 = 9 \ 3 \ 2 \mid 4 \ 5 \ 6 \ 7 \mid 1 \ 8.$$

Worked, child 2 (roles reversed; P_2 donates 8 2 6 5). The unmatched values are now 4 and 7: 4 maps via 8 to position 8, and 7 maps via 5, then via 2, to position 2. Filling from P_1 gives

$$\text{child}_2 = 1 \ 7 \ 3 \mid 8 \ 2 \ 6 \ 5 \mid 4 \ 9.$$

Always finish by checking that each of $1, \dots, 9$ appears exactly once.

3. Cycle crossover. This operator asks a different question: which genes are locked together by the two parents? Start at a position, take P_1 's value; find where P_2 's value at that position sits in P_1 ; move there; repeat until you return to the start. That closed loop is a *cycle*.

For our parents:

- **Cycle 1:** $1 \rightarrow 9 \rightarrow 4 \rightarrow 8 \rightarrow 1$, i.e. positions $\{1, 4, 8, 9\}$.
- **Cycle 2:** $2 \rightarrow 3 \rightarrow 7 \rightarrow 5 \rightarrow 2$, i.e. positions $\{2, 3, 5, 7\}$.
- **Cycle 3:** position $\{6\}$ alone, since both parents hold 6 there.

Now build offspring by taking whole cycles alternately from the two parents:

$$\text{child}_1 = 1 \ 3 \ 7 \ 4 \ 2 \ 6 \ 5 \ 8 \ 9, \quad \text{child}_2 = 9 \ 2 \ 3 \ 8 \ 5 \ 6 \ 7 \ 1 \ 4.$$

Because cycles are exchanged whole, validity is automatic, and every gene keeps a position it held in one of its parents.

4. Edge recombination. The three operators above all manipulate *gene values and positions*. For a routing problem such as TSP, what actually determines the cost is not where a city sits in the string but *which cities it is adjacent to* – the *edges*. Edge recombination therefore inherits edges.

Step 1: build the edge table. For each city, list its neighbours in both parents. Treat the tour as a closed circuit, so the last city is adjacent to the first. Mark with + any neighbour that appears in *both* parents (a common edge).

City	Neighbours	City	Neighbours
1	2, 9, 5, 4	6	5 ⁺ , 7, 2
2	1, 3, 8, 6	7	6, 8 ⁺ , 3
3	2, 4, 9, 7	8	7 ⁺ , 9, 2
4	3, 5, 1, 9	9	8, 1, 4, 3
5	4, 6 ⁺ , 1		

Step 2: construct the child one city at a time. Pick a random start; delete that city from every list; then choose the next city by these rules, in order:

1. if a common edge (+) is available from the current city, take it;
2. otherwise take the candidate whose own neighbour list is *shortest*;
3. break ties at random;
4. if the current city's list is empty, jump to any unused city at random.

Worked run, starting at city 1. From 1 the candidates are 2, 5, 4, 9 with remaining list lengths 3, 2, 3, 3, so 5 is shortest: take 5. From 5 the common edge 6^+ is available: take 6. From 6 the candidates 7 and 2 both have two entries, so break the tie at random; take 7. From 7 the common edge 8^+ is available: take 8. From 8 the candidates are 9 (two entries) and 2 (one entry): take 2. From 2 only 3 remains. From 3 the candidates 4 and 9 each have one entry; break the tie at random and take 4, then 9. The child is

1 5 6 7 8 2 3 4 9,

a valid permutation built from inherited edges. Note the two random tie-breaks: a different run would legitimately give a different child.

Why it is called constructive. Unlike the other operators, edge recombination does not copy blocks – it *builds* the solution one element at a time under a rule, which is how deterministic constructive methods work too.

7.7 Tree representation

When to use it. Some objects are naturally trees rather than strings: an arithmetic expression, a logical formula, a computer program, a finite-state machine. Evolving trees is the field of **genetic programming**, and the object being evolved is a *structure*, not a list of values.

Structure. A tree is built from two declared sets:

- the **function set** (internal, non-terminal nodes): the operators, e.g. $\{+, -, \times, \div\}$ or $\{\wedge, \vee, \neg, \rightarrow\}$;
- the **terminal set** (leaves): the variables and constants, e.g. $\{x, y, 2, 3, \pi\}$.

Defining these two sets is the representation decision. Reading the tree is just operator precedence: the root is the operation applied last.

Example. For $2\pi + ((x + 3) - \frac{y}{5+1})$, the root is +; its left subtree is $\times$ with leaves 2 and π ; its right subtree is $-$, whose left subtree is $+$ over x and 3, and whose right subtree is $\div$ over y and a $+$ node over 5 and 1. A logical formula such as $(x \wedge \text{true}) \rightarrow (\neg x \vee y)$ decomposes the same way, with logical connectives as the function set.

Variation on trees. Because the chromosome is a structure, the operators must vary the structure rather than a value.

- **Mutation:** choose a node at random and replace the subtree rooted there with a newly generated subtree – possibly just a single terminal.
- **Recombination:** choose a node in each parent and *exchange the two subtrees*. This is the direct analogue of exchanging substrings.

A consequence worth noticing: tree chromosomes are variable in size and non-linear, so unlike every other representation in this course, two chromosomes need not have the same length.

7.8 Compatibility summary

Type	Mutation	Recombination
Binary	Bit flip, per gene with probability p_m	One-point, multi-point, uniform
Integer	Creep ($\pm$ small step, risks premature convergence) or random resetting	One-point, multi-point, uniform
Real	Uniform reset in [LB, UB]; Gaussian $N(0, \sigma)$; self-adaptive (mutate σ first)	Single / simple / whole arithmetic (stays inside the parents' box); blend BLX- α (reaches outside); multi-parent
Permutation	Swap, insert, scramble, inversion	Order-1, PMX, cycle, edge recombination
Tree	Replace a subtree with a new random subtree	Exchange subtrees between parents

7.9 Checkpoint

- Can I name the five representations and a problem suited to each?
- Can I say why p_m is kept small, and what a very high p_m turns the GA into?
- Can I state the allele-frequency argument for why mutation is necessary?
- Can I explain creep mutation and premature convergence?
- Can I decode a bit string onto a real interval and state what sets the resolution?
- Can I say why σ must be mutated before x in self-adaptive mutation?
- Can I show that arithmetic crossover cannot leave the parents' bounding box, and how blend crossover escapes it?
- Can I execute order-1, PMX, cycle, and edge recombination on a given pair, and check validity afterwards?
- Can I define the function set and terminal set of a tree, and state both tree operators?

Source: ../resources/week-03-transcript.txt, permutation and tree representation segments. Corresponding worked-example slides are indexed in the public archive under worked-slide-examples/1.GA/ but were not used to derive the calculations above..

8 Quiz 1 scope and evidence

8.1 What is confirmed, and what is only supporting evidence?

This note prepares the Part 1 scope in `../resources/lecture-01-evolutionary-computing-part-1.pdf` together with the representation and operator material delivered in the Week 3 lecture. The supplied deck and transcripts establish a closed-book written Quiz 1 worth 10%, held in Week 4.

Logistics announced in the Week 3 lecture. Treat the following as announced in class and *verify every item in NTU Learn*, since the audio is ambiguous on the time of day and the seat allocation was still to be published.

- Written and closed book. Answers are written on the question paper itself, which is submitted in person and signed at the end.
- The cohort is split across three lecture theatres — LT19A, LT2A and LT5 — with the allocation to be announced through NTU’s official channel. Candidates must leave one empty seat between neighbours.
- Permitted: a pen and a photo ID (student card printout or passport). Mobile phones and smart devices are prohibited; do not use a phone to display the ID.
- The stated window is 9:30 to 10:30. Absence without a valid reason scores zero.

Scope announced in the Week 3 lecture. The quiz had not been set at the time of the announcement, and the lecturer stated it will be written so that it *does not* include the Week 4 lecture. The Week 4 session will finish a little further material and then review previous exam-style questions. On that basis, everything up to and including the Week 3 representation and operator material is examinable.

Confidence	Evidence	How it is used here
Current and authoritative	Lecture 1 Part 1 deck, PDF pages 1–36	Defines the examinable Part 1 concepts: classification, complexity, EC foundations, the EA loop and components, 8-queens, the simple GA example, EA behaviour, and optimisation context.
Current supporting context	Week 1 and Week 2 transcripts	Supports the assessment format and lecturer emphasis. Any tentative logistics remain unconfirmed.
Current and controlling	Week 3 transcript	Sole local source for metaheuristics, the no-free-lunch performance picture, deterministic versus heuristic search, neighbourhoods, and all five representations with their operators. The matching slide deck is not available locally.
Historical pattern only	Legacy quiz PDF pages 1–2; 2021, 2022, and 2024 walkthroughs	Suggests useful question forms involving classification, representation, encoding, fitness, and a hand-worked SGA generation. Answers below are newly derived.
Frequency evidence only	Exam-topic workbook, Sheet1 A1:G24	Shows recurring older-exam themes. It does not establish the present Quiz 1 scope.

Deliberately excluded. Differential evolution and particle swarm optimisation are outside this preparation pack, as are schema theory, niching, multi-objective optimisation, and machine learning. Note the change since the previous revision: the permutation and real-valued crossover families (order-1, PMX, cycle, edge recombination, arithmetic and blend crossover) were previously listed as excluded “advanced crossover families”. The Week 3 lecture taught them directly, so they are now *in* scope and are developed in Section 7.

Source: Lecture 1 Part 1 deck, PDF pp. 1–36; Week 1 and Week 2 transcripts; historical-pattern sources indexed in ../STATUS.md..

9 One-page memory core

Problem classification	For $y = M(x)$: unknown x means optimisation; unknown M means modelling/learning; unknown y means simulation.
Search language	Search space S : all encodable candidates. Feasible set $\mathcal{F} \subseteq S$: candidates satisfying hard constraints. Objective f : quality. CSP seeks any feasible candidate; COP seeks the best feasible candidate.
Complexity	P : solvable in polynomial time. NP : a proposed yes-certificate is verifiable in polynomial time. NP-complete: in NP and NP-hard. NP-hard: at least as hard as every NP problem, but may be an optimisation problem and need not be in NP.
EA cycle	Initialise $\rightarrow$ evaluate $\rightarrow$ select parents $\rightarrow$ vary by recombination/mutation $\rightarrow$ evaluate offspring $\rightarrow$ select survivors $\rightarrow$ test termination.
Representation	Phenotype is the domain solution; genotype is its encoding. A useful encoding makes intended solutions reachable and lets operators preserve or repair validity.
Selection	Fitness-proportionate selection: $p_i = f_i / \sum_j f_j$ when fitnesses are non-negative and larger is better. Selection favours; it does not guarantee.
8-queens	Permutation $q = (q_1, \dots, q_N)$ places one queen in column i , row q_i . Queens i, j conflict diagonally when $ q_i - q_j = i - j $. With C diagonal conflicts, $f = \binom{N}{2} - C$.
Binary decoding	For bits $b_{L-1} \dots b_0$, $z = \sum_{k=0}^{L-1} 2^k b_k$. Optional interval map: $x = a + \frac{z}{2^L - 1}(b - a)$.
Mutation checks	Across N strings of length L , expected flipped bits are $N L p_m$. Probability of no flip is $(1 - p_m)^{NL}$.
Optimisation context	A neighbourhood defines local moves. A local optimum is best only in its neighbourhood; a global optimum is best in the whole feasible set. No single optimiser is best across all possible problems.
Representation first	Choose the representation before the operators, because the operators must suit the representation . Five types: binary $\{0, 1\}^L$; integer $\{1, \dots, K\}^L$; real $\mathbb{R}^n$; permutation of $1, \dots, L$; tree (function set + terminal set).
Binary operators	Bit flip with small p_m (guide: between $1/\mu$ and $1/L$; a large p_m degenerates to random search). Crossover: one-point, multi-point, uniform. Only crossover combines parental information; only mutation introduces new information.
Integer operators	Creep ($\pm$ small step, causes premature convergence) or random resetting. Crossovers as for binary. Graph colouring: $c \in \{1, \dots, k\}^L$, minimise $\sum_{(u,v) \in E} \mathbf{1}[c_u = c_v]$; $k = 3$ fails on the US map, $k = 4$ always suffices.
Real operators	Decode $\Gamma(a) = x + \frac{y-x}{2^L-1} \sum a_i 2^{L-i}$ (L sets resolution). Mutation: uniform reset, or $x'_i = x_i + N(0, \sigma)$; self-adaptive puts σ in the chromosome and <i>mutates</i> σ <i>before</i> x . Crossover: $z_i = \alpha x_i + (1 - \alpha) y_i$ stays inside the parents' box; blend samples $[x_i - \alpha d_i, y_i + \alpha d_i]$ with $d_i = y_i - x_i$ and escapes it.
Permutation operators	Every operator must return a permutation. Mutation: swap, insert, scramble, inversion (inversion preserves adjacency). Crossover: order-1, PMX, cycle, edge recombination (edge-based, constructive, uses common edges then shortest list then random).
Tree operators	Mutation replaces a subtree with a new random subtree; recombination exchanges subtrees. Chromosomes are variable length.

Closed-book recall test. Cover the table and reconstruct the three $y = M(x)$ cases, the P/NP relationships, the seven EA stages, the 8-queens conflict test, roulette selection, binary decoding, the five representations with one legal operator pair each, and the no-free-lunch conclusion.

10 Worked archetypes

10.1 Archetype 1: classify the problem before choosing an algorithm

Question. A factory knows a simulator M that predicts energy use y for a machine setting x . Classify each task:

- (a) Find a setting that minimises energy while meeting production constraints.
- (b) Predict the energy used by one specified setting.
- (c) Infer a predictive relationship from logged settings and measured energy.

Construction. Write $y = M(x)$ and ask which object is unknown.

- (a) M is known and the desired low-energy outcome is specified, but x is unknown: **optimisation**. The search space contains encodable settings, the feasible set satisfies production constraints, and the objective is predicted energy.
- (b) x and M are known, while y is requested: **simulation**.
- (c) Input–output examples are known, while M is unknown: **modelling/learning**.

Trap. The use of a computer model does not automatically make a task simulation. Classification depends on what is unknown.

10.2 Archetype 2: reason correctly about P, NP, NP-complete, and NP-hard

Question. Assess the claim that NP means non-polynomial and therefore an NP-hard optimisation problem cannot be checked efficiently.

Answer. The claim combines two errors.

1. NP means *nondeterministic polynomial time*, commonly characterised by polynomial-time verification of a proposed yes-certificate for a decision problem. It does not mean that the problem is proved to require non-polynomial time.
2. NP-hard is a hardness relation. An NP-hard problem need not itself be a decision problem or belong to NP. For an optimisation problem, checking that a candidate is feasible and calculating its objective may be easy even when proving that it is globally optimal is hard.

If a decision problem is both in NP and NP-hard, it is NP-complete. Every problem in P is also in NP. Whether $\mathbf{P} = \mathbf{NP}$ remains unresolved.

10.3 Archetype 3: trace the evolutionary algorithm loop

Question. A population contains four binary strings. Describe one generation without confusing parent selection and survivor selection.

1. **Represent and initialise:** choose an encoding and create the initial population.
2. **Evaluate:** decode each genotype, calculate its objective, and convert it to the fitness used by selection.
3. **Select parents:** sample candidates for reproduction. Better fitness normally means greater probability, not certainty.
4. **Recombine:** exchange genetic material between selected parents to make offspring.
5. **Mutate:** randomly change genes, supplying new variation.

6. **Evaluate offspring:** decode and score the new candidates.
7. **Select survivors:** decide which parents and/or offspring form the next population.
8. **Terminate or repeat:** stop at a target, budget, or stagnation condition; otherwise begin the next generation.

Trap. Parent selection answers who reproduces; survivor selection answers who remains. They can use different rules.

10.4 Archetype 4: represent and score 8-queens

For $N = 4$, let q_i be the row of the queen in column i . The permutation $q = (2, 4, 1, 3)$ automatically gives one queen in each row and column. For every pair $i < j$, compare $|q_i - q_j|$ with $|i - j|$:

(i, j)	(1, 2)	(1, 3)	(1, 4)	(2, 3)	(2, 4)	(3, 4)
$ q_i - q_j $	2	1	1	3	1	2
$ i - j $	1	2	3	1	2	1

No pair is equal, so $C = 0$ and $f(q) = \binom{4}{2} - C = 6$. A swap mutation exchanging the last two positions gives $q' = (2, 4, 3, 1)$. It remains a valid permutation, but pair (2, 3) conflicts, so $C = 1$ and $f(q') = 5$.

Operator lesson. Swap mutation preserves the permutation. Recombination must likewise preserve it or be followed by repair.

10.5 Archetype 5: calculate one complete SGA generation

Maximise $f(x) = x^2$ for an unsigned four-bit integer. Start with:

chromosome	x	$f(x)$	$p_i = f_i / \sum f_j$
0011	3	9	$9/311 = 0.0289$
0101	5	25	$25/311 = 0.0804$
1001	9	81	$81/311 = 0.2605$
1110	14	196	$196/311 = 0.6302$

The fitness sum is 311. Suppose roulette draws produce the mating pool [1110, 1001, 1110, 0101]. Pair adjacent parents and cross after bit 2:

$$\begin{aligned} 11|10, 10|01 &\longrightarrow 11|01 = 1101, & 10|10 &= 1010, \\ 11|10, 01|01 &\longrightarrow 11|01 = 1101, & 01|10 &= 0110. \end{aligned}$$

Suppose mutation flips the final bit of the fourth offspring only, so $0110 \rightarrow 0111$. With generational replacement the next population is [1101, 1010, 1101, 0111], decoding to [13, 10, 13, 7], with fitnesses [169, 100, 169, 49]. Its total fitness is 487 and its best fitness is 169.

Checks. The roulette outcomes and mutation event are sampled events, not guarantees. Decode after variation, recompute fitness, and use the correct generation's fitness sum.

10.6 Archetype 6: choose a representation and legal operators

Task. A courier must visit 12 drop-off points once each and return to the depot at minimum distance. State a suitable representation, one mutation and one crossover operator, and justify both.

Answer. The unknown is an *order*, so use a permutation of $1, \dots, 12$ where gene position gives visit order; the genotype space is the $12!$ orderings. Mutation: inversion, which reverses

a segment and so changes the order while preserving most adjacency (i.e. most edges), which is what the distance objective actually depends on. Crossover: edge recombination, because the cost is a sum over edges and this operator inherits edges rather than positions. Ordinary one-point crossover is rejected: it would repeat some drop-offs and omit others, producing an invalid tour.

Marking logic. Name the representation *and* the genotype space; give one legal operator of each kind; justify against the objective; state explicitly why a binary-style crossover is invalid here.

10.7 Archetype 7: real-valued crossover arithmetic

Task. Parents $x = (0.3, 0.7, 0.8, 0.2)$ and $y = (0.5, 0.1, 0.2, 0.9)$. (a) Apply whole arithmetic crossover with $\alpha = 0.3$. (b) State the range of values the first gene of any arithmetic child can take. (c) Give that range under blend crossover with $\alpha = 0.5$.

Answer. (a) $z^{(1)} = 0.3x + 0.7y = (0.44, 0.28, 0.38, 0.69)$ and $z^{(2)} = 0.7x + 0.3y = (0.36, 0.52, 0.62, 0.41)$. Check: $z^{(1)} + z^{(2)} = x + y = (0.8, 0.8, 1.0, 1.1)$. (b) $z_1 = \alpha(0.3) + (1 - \alpha)(0.5)$ with $\alpha \in [0, 1]$, so $z_1 \in [0.3, 0.5]$: strictly between the parents. (c) With $x_1 = 0.3 < y_1 = 0.5$, $d_1 = 0.2$, so the child is sampled from $[0.3 - 0.5(0.2), 0.5 + 0.5(0.2)] = [0.2, 0.6]$.

Marking logic. Correct weighted averages; the sum check; an explicit statement that arithmetic offspring cannot leave the parental interval while blend offspring can.

10.8 Archetype 8: permutation crossover under exam conditions

Task. With $P_1 = 1\ 2\ 3\ 4\ 5\ 6\ 7\ 8\ 9$ and $P_2 = 9\ 3\ 7\ 8\ 2\ 6\ 5\ 1\ 4$ and segment positions 4–7, produce one PMX child and one cycle-crossover child, and verify validity.

Answer. PMX (segment from P_1): $9\ 3\ 2\ 4\ 5\ 6\ 7\ 1\ 8$ — the unmatched values 2 and 8 are relocated by following the mapping chains $2 \rightarrow 5 \rightarrow 7$ and $8 \rightarrow 4$, and the remaining positions are filled from P_2 . Cycle crossover: the cycles are $\{1, 4, 8, 9\}$, $\{2, 3, 5, 7\}$ and $\{6\}$; taking cycles alternately gives $1\ 3\ 7\ 4\ 2\ 6\ 5\ 8\ 9$. Both contain each of $1, \dots, 9$ exactly once.

Marking logic. Show the mapping chain or the cycle decomposition, not just the final string; finish with an explicit validity check. A child with a repeated value scores zero regardless of method.

10.9 Archetype 9: binary encoding of a real variable

Task. A variable lies in $[0.25, 1.88]$ and is encoded in $L = 4$ bits. Decode 1010, state the resolution, and state the effect of moving to $L = 10$.

Answer. $\sum a_i 2^{4-i} = 8 + 2 = 10$, so

$$\Gamma = 0.25 + \frac{1.88 - 0.25}{2^4 - 1}(10) = 0.25 + \frac{1.63}{15}(10) = 1.3367.$$

The resolution is $1.63/15 = 0.1087$ per code step. At $L = 10$ it becomes $1.63/1023 \approx 0.00159$: finer accuracy, at the cost of a genotype space of 2^{10} instead of 2^4 and correspondingly slower evolution.

Marking logic. Use $2^L - 1$, not 2^L , in the denominator; state that L buys resolution and charges search cost; note that a real interval is infinite while the code space is finite.

11 Progressive drills

Attempt all twenty without notes. Hints are brief; complete solutions begin on the next page.

- D1.** For $y = M(x)$, classify: (i) estimating M from examples; (ii) finding x for a desired y with known M ; (iii) finding y for known x, M .

Hint: identify the unknown object.

- D2.** A delivery problem has all possible route encodings S , a maximum-distance rule, and total fuel use f . Identify the search space, feasible set, and objective.

Hint: constraints define a subset of the search space.

- D3.** How many ordered routes are there through eight labelled cities before symmetry reductions? Why does this number matter?

Hint: order all eight objects.

- D4.** Correct the claim that every NP-hard problem is an NP-complete decision problem and NP means non-polynomial.

Hint: separate membership from hardness.

- D5.** Put these in a valid EA order: mutation, termination test, initialisation, offspring evaluation, parent selection, survivor selection, population evaluation, recombination.

Hint: candidates must exist before evaluation.

- D6.** Three candidates have fitnesses 2, 3, 5. Calculate their roulette-selection probabilities.

Hint: divide by total fitness.

- D7.** Explain in one sentence why repeatedly selecting only the current best candidate can be risky.

Hint: think about diversity and local optima.

- D8.** For $q = (2, 4, 1, 3)$ in 4-queens, calculate the diagonal conflicts and fitness $\binom{4}{2} - C$.

Hint: test every pair.

- D9.** Parents (1, 2, 3, 4) and (4, 3, 2, 1) undergo ordinary one-point crossover after position 2. Write the children and explain the validity problem.

Hint: a permutation contains each row once.

- D10.** Decode the unsigned binary string 10110.

Hint: use place values 16, 8, 4, 2, 1.

- D11.** Map 10110 linearly to $[10, 20]$, including both endpoints.

Hint: use $10 + \frac{z}{31}(20 - 10)$.

- D12.** Six strings each have length eight and independent bit-mutation probability $p_m = 0.005$. Find the expected flips and probability of no flips.

Hint: there are 48 trials.

- D13.** State the two strands of natural inspiration behind GAs, and define “metaheuristic” in one sentence.

Hint: one strand is about competition, the other about encoding.

- D14.** Sketch the no-free-lunch performance picture and state the conclusion it supports, plus one thing it does not claim.
Hint: compare areas under the curves.
- D15.** A binary population starts with zeros and ones in a 1:1 ratio. What is the ratio after 100 generations of crossover alone, and what does this prove?
Hint: does crossover create new allele values?
- D16.** Name the two mutation options for an integer representation and state the failure mode of the first.
Hint: small steps accumulate.
- D17.** Decode $a = 1010$ onto $[0.25, 1.88]$ with $L = 4$, and give the step size.
Hint: the denominator is $2^L - 1$.
- D18.** In self-adaptive mutation the chromosome is $\langle x_1, \dots, x_n, \sigma \rangle$. Which is mutated first, and why?
Hint: what does fitness actually measure?
- D19.** Parents $x = (0.3, 0.7)$ and $y = (0.5, 0.1)$. Give both whole-arithmetic children for $\alpha = 0.5$, and say what is wrong with the result.
Hint: compute both, then compare them.
- D20.** With $P_1 = 1\ 2\ 3\ 4\ 5\ 6\ 7\ 8\ 9$, $P_2 = 9\ 3\ 7\ 8\ 2\ 6\ 5\ 1\ 4$ and segment positions 4–7, give the order-1 crossover child.
Hint: fill from position 8 onward, wrapping, skipping duplicates.

12 Solutions to the progressive drills

- D1.** (i) Modelling/learning; (ii) optimisation; (iii) simulation.
- D2.** S is every encodable route; $\mathcal{F}$ is the subset obeying the maximum-distance rule; minimise total fuel use f .
- D3.** $8! = 40320$. Factorial growth makes exhaustive enumeration rapidly expensive.
- D4.** NP is polynomial-time verification of yes-certificates for decision problems, not non-polynomial. NP-hard problems need not be in NP or be decision problems. NP-complete problems are both in NP and NP-hard.
- D5.**
 initialise $\rightarrow$ evaluate population $\rightarrow$ select parents $\rightarrow$ recombine $\rightarrow$ mutate
 $\rightarrow$ evaluate offspring $\rightarrow$ select survivors $\rightarrow$ test termination.
- D6.** Total 10; probabilities 0.2, 0.3, 0.5, summing to 1.
- D7.** It can destroy diversity and cause premature convergence around a local optimum.
- D8.** No conflicting pair, so $C = 0$ and $f = 6$.
- D9.** Children (1, 2, 2, 1) and (4, 3, 3, 4) repeat and omit rows; use a permutation-preserving crossover or repair.
- D10.** $10110_2 = 16 + 4 + 2 = 22$.
- D11.** $x = 10 + \frac{22}{31}(10) = 17.0968$ approximately.
- D12.** Expected flips $= 6(8)(0.005) = 0.24$; no-flip probability $= 0.995^{48} \approx 0.7862$.
- D13.** Darwinian evolution (limited resources cause competition; selection retains the fitter; reproduction rewards survival) and genetics (DNA as an encoded description whose expression is the phenotype). A metaheuristic is a general, higher-level search scheme built from rules of thumb — here rules drawn from nature rather than from problem-specific human knowledge.
- D14.** Performance against all problems: random search flat and low; a custom method a narrow tall spike; a plain EA broad and moderately high; each added increment of domain knowledge raises the peak and narrows the range. Equal areas imply equal average performance over all problems, hence no universally best optimiser. It does *not* claim that algorithm choice is irrelevant on a particular real problem.
- D15.** Still 1:1. Crossover only redistributes existing alleles between strings, so it cannot restore a lost value; this is why mutation, which introduces new information, is necessary.
- D16.** Creep (add a small $\pm$ step with probability p) and random resetting. Creep moves individuals in small steps, so the population drifts together and converges prematurely.
- D17.** $\sum a_i 2^{4-i} = 10$, so $\Gamma = 0.25 + \frac{1.63}{15}(10) = 1.3367$; the step size is $1.63/15 = 0.1087$.
- D18.** Mutate σ first. Fitness is evaluated on x only, so σ' can only be judged through the quality of the x' it generated; mutating σ first makes that dependency hold.
- D19.** Both children are (0.40, 0.40). At $\alpha = 0.5$ the two arithmetic children are identical, so the operator returns one distinct point instead of two.
- D20.** 3 8 2 4 5 6 7 1 9: segment 4 5 6 7 is kept, and P_2 's remaining values in order from position 8 are 1, 9, 3, 8, 2, filling positions 8, 9, 1, 2, 3.

13 Mock Quiz A: foundations and a hand-worked generation

Conditions: 60 minutes, closed book, 100 marks.

Question 1: Problem formulation (20 marks; 10 minutes)

A trained surrogate predicts drag $y = M(x)$ from an airfoil design x .

- (a) Classify finding the lowest-drag design subject to strength constraints. Identify S , $\mathcal{F}$, and the objective. [10]
- (b) Classify (i) predicting drag for a given design and (ii) learning the surrogate from examples. [6]
- (c) Distinguish a CSP from a constrained optimisation problem. [4]

Question 2: Complexity and EC foundations (20 marks; 10 minutes)

- (a) Define P, NP, NP-complete, and NP-hard. [12]
- (b) Explain why NP-hardness motivates heuristic search but does not prove an EA will perform well. [4]
- (c) Distinguish genotype from phenotype, with one example. [4]

Question 3: The EA loop (20 marks; 10 minutes)

- (a) Write one complete generation of a general EA in a correct order. [12]
- (b) Distinguish parent selection from survivor selection. [4]
- (c) Give two sensible termination conditions. [4]

Question 4: 4-queens representation (20 marks; 12 minutes)

Use q_i = row in column i and fitness $f = \binom{4}{2} - C$.

- (a) For $q = (1, 3, 4, 2)$, list every diagonal conflict and calculate f . [10]
- (b) Explain which constraints the permutation enforces automatically. [4]
- (c) Explain why ordinary one-point crossover may fail and give a remedy. [6]

Question 5: Simple GA calculation (20 marks; 18 minutes)

Maximise $f(x) = x^2$ using three-bit unsigned strings. The population is 010, 011, 101, 110.

- (a) Decode, calculate all fitnesses, and calculate roulette probabilities. [10]
- (b) The mating pool is 110, 101, 011, 110. Cross adjacent parents after bit 1. [6]
- (c) Flip the last bit of the first child only. Give the next population and best fitness. [4]

14 Mock Quiz A: marking guide

Q1 (20). Optimisation (2); S all encodable designs (2); $\mathcal{F}$ strength-feasible designs (2); minimise drag (2); known M /unknown x (2). Simulation (3), modelling/learning (3). CSP seeks any feasible candidate (2); constrained optimisation seeks the best (2).

Q2 (20). P solvable in polynomial time (3); NP has polynomially verifiable yes-certificates (3); NP-complete is in NP and NP-hard (3); NP-hard need not be in NP (3). Motivation without algorithm-specific guarantee (4). Genotype is encoding; phenotype is domain solution, with example (4).

Q3 (20). Initialise, evaluate, parent select, recombine, mutate, evaluate offspring, survivor select, terminate/repeat: 12. Parent selection chooses reproducers; survivor selection forms the next population: 4. Any two of target, evaluation/time budget, or stagnation: 4.

Q4 (20). Only (2,3) conflicts (6); $C = 1$, $f = 5$ (4). One queen per column and row (4). Ordinary crossover can duplicate/omit rows (3); permutation-preserving crossover or repair (3).

Q5 (20). Decodings 2, 3, 5, 6 (2); fitnesses 4, 9, 25, 36 (2); sum 74 (2); probabilities $4/74$, $9/74$, $25/74$, $36/74$ (4). The crosses are

$$1|10, 1|01 \rightarrow 101, 110 \quad \text{and} \quad 0|11, 1|10 \rightarrow 010, 111$$

(6). Mutate $101 \rightarrow 100$; the next population is 100, 110, 010, 111 (2), and its best fitness is 49 (2).

Total: 100 marks.

15 Mock Quiz B: representation, operators, and optimisation context

Conditions: 60 minutes, closed book, 100 marks.

Question 1: Representation and encoding (20 marks; 12 minutes)

- (a) Define chromosome, gene, allele, genotype, and phenotype. [10]
- (b) Decode 11010_2 and map it linearly to $[-2, 3]$, including endpoints. [6]
- (c) State two properties of a useful representation. [4]

Question 2: Selection and variation (20 marks; 10 minutes)

Four candidates have fitnesses 1, 2, 3, 4.

- (a) Calculate roulette probabilities. [6]
- (b) Explain why the best candidate is not guaranteed to be selected. [4]
- (c) Distinguish recombination from mutation and state one role of each. [6]
- (d) State one danger of excessive selection pressure. [4]

Question 3: 5-queens fitness (20 marks; 12 minutes)

Use $q = (1, 2, 3, 5, 4)$ and fitness $f = \binom{5}{2} - C$.

- (a) Identify all diagonal conflicts. [12]
- (b) Calculate C and f . [4]
- (c) Explain why swap mutation preserves row/column feasibility. [4]

Question 4: SGA checks (25 marks; 16 minutes)

Five strings have length six; bit mutation is independent with $p_m = 0.01$.

- (a) Find expected flips and probability of no flips. [8]
- (b) Cross $101|011$ and $010|110$ at the shown point. [5]
- (c) Decode both children as unsigned integers. [4]
- (d) For minimising $g(x) = (x - 20)^2$, explain why raw g is wrong in a larger-is-better roulette rule and give a valid transformation. [8]

Question 5: Search behaviour (15 marks; 10 minutes)

- (a) Distinguish local and global optima using a neighbourhood. [6]
- (b) Explain exploration versus exploitation. [5]
- (c) State the practical message of no free lunch. [4]

16 Mock Quiz B: marking guide

Q1 (20). Chromosome/genotype, gene/component, allele/value, genotype/encoding, phenotype/domain solution: 2 each. $11010_2 = 26$ (2); $x = -2 + \frac{26}{31}(5) = 2.1935$ (4). Any two useful representation properties: 2 each.

Q2 (20). Total 10, probabilities 0.1, 0.2, 0.3, 0.4 (6). Stochastic selection is not certainty (4). Recombination combines parental material (3); mutation supplies random new variation (3). Excess pressure can destroy diversity and cause premature convergence (4).

Q3 (20). Conflicts (1, 2), (1, 3), (2, 3), (4, 5): 3 each. Thus $C = 4$, $f = 10 - 4 = 6$ (4). Swapping retains each row once, while columns are positions (4).

Q4 (25). There are 30 bits; expected flips 0.3 (4); no-flip probability $0.99^{30} \approx 0.7397$ (4). Children 101110 and 010011 (5), decoding to 46 and 19 (4). Raw g rewards larger error (4); for example $F = 1/(1 + g)$ is positive and decreases with error (4).

Q5 (15). Local is best in its neighbourhood; global is best in the whole feasible set (6). Exploration samples new regions; exploitation improves promising regions (5). No optimiser dominates all possible problems, so problem knowledge and empirical testing matter (4).

Total: 100 marks.

17 Seven-day preparation schedule

Days are counted back from the quiz, because the exact slot must still be confirmed in NTULearn. The Week 3 material carries the most new content and gets two full days.

Day	Focus	Minimum deliverable
D-7	Scope, classification, complexity	Reproduce the scope map; solve D1-D4 without notes.
D-6	EA loop, components, 8-queens	Draw the complete loop from memory; solve D5-D9.
D-5	SGA arithmetic and binary coding	Recalculate Archetype 5; solve D10-D12 and D17; check every sum.
D-4	EA in context and NFL	Sketch the four performance curves from memory; solve D13-D15; state what NFL does not claim.
D-3	Representations I: binary, integer, real	Recalculate Archetypes 7 and 9; solve D16-D19; reproduce the compatibility table.
D-2	Representations II: permutation and tree	Recalculate Archetype 8 by hand; solve D20; run order-1, PMX, cycle and edge recombination on one fresh pair and verify validity each time.
D-1	Timed mocks and weak spots	Attempt Mock A and Mock B under time; mark in a different colour; write a one-line cause for each error; stop heavy study early.

17.1 Quiz-day checklist

- Verify the official date, time, and lecture-theatre allocation (LT19A, LT2A or LT5) in NTULearn.
- Bring only a pen and a photo ID; a printed student card or a passport is acceptable. Leave phones and smart devices aside — do not plan to show the ID on a phone.
- Leave one empty seat between yourself and each neighbour.
- Answers are written on the question paper; submit it in person and sign at the end.
- Before calculating, write the encoding, objective direction, and fitness convention.
- For roulette selection, calculate the fitness sum and check probabilities sum to 1.
- After crossover or mutation, check chromosome length and representation validity; for a permutation, confirm every value appears exactly once.
- For real-valued crossover, remember arithmetic children stay inside the parents' interval and blend children may fall outside it.
- In self-adaptive mutation, mutate σ before x .
- For 8-queens, test each pair once using $i < j$; do not double-count.
- For complexity questions, do not expand NP as non-polynomial.
- Reserve the final five minutes for arithmetic and unanswered subparts.

Source: .. /resources/lecture-01-evolutionary-computing-part-1. pdf, PDF pp. 1-36, and .. /resources/week-03-transcript. txt in full. Historical sources were used only to design original question patterns; all calculations and solutions here were independently derived..